
Hardware/Software Design Description

Robotic Building Block Collection System

Version 1.0 (Approved)

Group 7

Date: 26/5/2026

Revision History

Date	Reason For Changes	Version
2025-09-15	Initial release. Established base architecture, implementing YOLO object detection and initial manipulator-based collection logic.	1.0
2026-04-08	Hardware redesign. Replaced the mechanical arm with a fixed shovel structure for scoop-based collection to improve reliability.	2.0
2026-04-15	ROS 2 bridge implementation. Deployed the driver interface for the RoboMaster chassis, enabling seamless underlying hardware control.	2.1
2026-05-01	Navigation deployment. Integrated 2D SLAM for mapping and configured Nav2 for autonomous global path planning and obstacle avoidance.	2.2
2026-05-10	Control upgrade. Integrated Behavior Tree (BT.CPP) for the Hierarchical Center-to-Local Sweeping strategy and strict exception handling.	2.3
2026-05-16	Visual servoing optimization. Added multi-frame debouncing, EMA filtering, and blind-zone takeover mechanisms for robust alignment.	2.4
2026-05-18	Final integration. Added a 3D-printed anti-slip wedge to the shovel. Achieved full closed-loop operation with AMCL-based return-to-home.	3.0

Contents

1	Introduction	4
1.1	Project Background and Objectives	4
1.2	Problem Definitions	5
1.3	Organization of Document	7
2	Hardware Architecture	8
2.1	Experimental Hardware Components	8
2.1.1	RoboMaster EP Mobile Robot Chassis	8
2.1.2	NVIDIA Jetson Orin NX Edge AI Computing Unit	9
2.1.3	YDLIDAR X3 PRO LiDAR	9
2.1.4	Interface Cables and Power Connection Materials	9
2.1.5	3D Printing Material	9
2.2	Task Challenges, Limitations of the Original Mechanism, and Proposed Design Changes	10
2.2.1	Key Task Challenges	10
2.2.2	Limitations of the Original Mechanical Structure	10
2.2.3	Targeted Hardware Modifications	11
2.3	Mechanical Modeling, 3D Printing, and Hardware Installation	12
2.3.1	CAD Modeling and 3D Printing	12
2.3.2	Hardware Installation Procedure	12
2.3.3	Summary of Component Geometric and Inertial Parameters	13
2.3.4	Coordinate Transforms and TF Tree	14
2.3.5	Visualization in RViz2	16
2.3.6	Summary	16
3	ROS Communication and YOLO Perception	17
3.1	Vision and Ranging Sensors	17
3.1.1	Camera Selection and Setup	17
3.2	Onboard Computing Requirements	17
3.2.1	Hardware Specifications	18
3.3	ROS Communication	18
3.3.1	System Objectives	18

3.3.2	Architectural Principles	19
3.3.3	Implementation Steps	19
3.4	YOLO Training and Deployment	20
3.4.1	Algorithmic Principles	20
3.4.2	Training Steps	20
3.4.3	Training Result	21
3.5	ROS Vision Node Integration	21
4	SLAM and Navigation	22
4.1	Brief Introduction	22
4.2	Integration	23
4.2.1	Environment Integration	23
4.3	Simulation	25
5	Map Partition and Collect Algorithm	27
5.1	Map Partitioning	27
5.1.1	Pre-processing	27
5.1.2	Seed Point Extraction	29
5.1.3	Voronoi Partitioning	30
5.1.4	Geometry Center	33
5.2	Collection Algorithm	34
5.2.1	Hierarchical Center-to-Local Sweeping Strategy	35
5.2.2	Robust Visual Servoing and Fault Tolerance	35
6	Field Experiment and Analysis	37
6.1	SLAM and Navigation	37
6.1.1	Mapping	38
6.1.2	Localization	38
6.1.3	Navigation	38
6.2	Map partition	39
6.3	Analysis	40
6.3.1	Scenario 1: Single-Block across Multiple Regions	40
6.3.2	Scenario 2: Multi-Block within a Single Region	41
7	Conclusion	42

Chapter 1

Introduction

1.1 Project Background and Objectives

This project develops an autonomous building block collection and storage robot based on the DJI RoboMaster EP platform. It addresses a common household problem: **scattered building blocks are time-consuming to collect manually, occupy indoor space, may block walking paths, and can create safety risks for children and elderly people.** The objective is therefore to enable a mobile robot to search for, collect, and store building blocks indoors with minimal human intervention.

The original RoboMaster EP platform alone is not sufficient for this task due to several hardware and software limitations. **A block-collecting robot must know its own position, navigate to target areas, and autonomously return to the charging station.** However, the original platform lacks a complete mapping, localization, and autonomous navigation solution. Furthermore, the initial gripper-based collection method is inefficient, as picking up blocks one by one causes repeated travels, increasing task durations and power consumption.

To solve the above difficulties, we propose a comprehensive hardware-software co-design solution. Our core technical contributions are structured as follows:

- **Hardware Modification for High-Efficiency Collection:** A customized double-layer mounting frame was designed in SolidWorks to integrate a LiDAR sensor and an NVIDIA Jetson Orin NX board. To revolutionize collection efficiency, we replaced the traditional gripper with a combined **scoop bucket and L-shaped mechanical arm mechanism**, enabling the robot to sweep multiple blocks of diverse shapes into the container in a single operation.
- **ROS 2 Integration and AI Perception:** The open-source `robomaster_ros` package bridges the RoboMaster SDK with the ROS 2 ecosystem. Building block detection is powered by a fine-tuned **YOLOv8** model deployed on the Jetson edge computing unit.
- **SLAM and Autonomous Navigation:** A 2D laser SLAM (*Simultaneous Localization And Mapping*) pipeline is utilized to construct the indoor occupancy grid map, while

the Nav2 framework and **AMCL** provide real-time, robust obstacle-aware navigation.

- **Innovative Collection Algorithm:** Instead of using traditional Complete Coverage Path Planning (CCPP), we introduce a **Watershed-based Generalized Voronoi Partitioning** algorithm to divide the open environment into focused, obstacle-free sub-regions. A **Behavior Tree (BT)** architecture is then employed to execute a highly fault-tolerant "center-to-local" sweeping strategy, ensuring exhaustive clearance.

The project is open-source at <https://github.com/KunLiangChen/BrickPick>, with the development process documented through the Notion page linked in the GitHub README.

1.2 Problem Definitions

This section presents a precise formulation of the problem addressed in the **Robotic Building Block Pick System** report. To clearly define the scope and constraints, we first establish the basic assumptions followed by the formal problem definition.

Assumptions 1 (Fixed Scene). *The task typically takes place in a fixed indoor environment. Therefore, repetitive SLAM (Simultaneous Localization and Mapping) is not required; the robot operates based on a known or pre-built map.*

Assumptions 2 (Clustered Distribution). *The target blocks generally exhibit clustering characteristics. Discrete individual blocks are assumed to be distributed within a reasonable distance from the main clusters.*

Assumptions 3 (Open space). *The system focuses on open areas. Retrieving blocks from extremely narrow or occluded spaces is outside the scope of this project.*

Given a specific area containing multiple static obstacles, several piles of building blocks, and discrete scattered blocks, the objective is for the robot to:

Problem (Building Block Collection Task). *The objective is to define a mission sequence for a mobile robot in an obstacle-cluttered environment to: (i) traverse safely without collisions; (ii) retrieve all clustered and scattered blocks; and (iii) execute an autonomous return-to-home maneuver at the charging dock.*

Abbreviation Explanation

Term	Definition
AMCL	Adaptive Monte Carlo Localization, a probabilistic localization system for a robot moving in 2D.

BT	Behavior Tree, a mathematical model of plan execution used to structure the hierarchical and fault-tolerant collection logic.
CAD	Computer-Aided Design, the technology used to model the custom mechanical structures.
CCPP	Complete Coverage Path Planning, a traditional algorithm for traversing an entire area, which this project optimizes against.
CMOS	Complementary Metal Oxide Semiconductor, the image sensor type used in the system camera.
CUDA	Compute Unified Device Architecture, NVIDIA's parallel computing platform for accelerating AI inference.
EDT	Euclidean Distance Transform, an algorithm used in preprocessing to calculate the minimum distance to the nearest obstacle.
EMA	Exponential Moving Average, a filtering technique used during the dynamic approach to smooth tracking coordinates and prevent jitter.
FOV	Field of View, the angular range of the observable area of the camera and infrared sensor.
FSM	Finite State Machine, a traditional mathematical model of computation, which this project replaces with Behavior Trees for superior modularity.
GVD	Generalized Voronoi Diagram, a spatial partition method used to segment the map while respecting obstacle boundaries.
I/O	Input/Output, the data interaction process between the system modules and external devices.
LiDAR	Light Detection and Ranging, a remote sensing method that uses light in the form of a pulsed laser to measure variable distances.
NMS	Non-Maximum Suppression, a post-processing technique in YOLO used to filter out duplicate overlapping detection boxes.
OGM	Occupancy Grid Map, a grid-based representation of the environment used by SLAM and Nav2.
OpenCV	Open Source Computer Vision Library, the core library used in the system for image processing and video stream operations.
PLA	Polylactic Acid, the thermoplastic monomer used as the 3D printing material for the customized hardware components.
ROI	Region of Interest, a specifically selected image area for targeted visual processing.

ROS	Robot Operating System, a flexible framework for writing robot software used to bridge hardware and algorithms.
SDK	Software Development Kit, the RoboMaster development kit used for robot chassis, gripper and vision module control.
SLAM	Simultaneous Localization and Mapping, used to construct an indoor map of an unknown environment.
TF	Transform Framework, a system in ROS that lets the user keep track of multiple coordinate frames over time.
TOPS	Tera Operations Per Second, the unit used to measure the AI computing performance.
URDF	Unified Robot Description Format, an XML format for representing the robot model within ROS.
YOLO	You Only Look Once, a real-time object detection algorithm; the system uses YOLOv8 for building block detection.

1.3 Organization of Document

This document is organized as follows:

- **Chapter 1: Introduction** Clarifies the purpose and scope of this design document, defines core terms and acronyms, and explains the document structure.
- **Chapter 2: Hardware Architecture** Details the physical components and customized 3D-printed modifications, including the elevated LiDAR mount, bucket, and L-shaped sweeping mechanism.
- **Chapter 3: ROS Communication and YOLO Perception** Explains the integration of the NVIDIA Jetson Orin NX, the ROS 2 software bridge, and the deployment of the YOLOv8 model for real-time block recognition.
- **Chapter 4: SLAM and Navigation** Describes the 2D laser mapping process using *slam toolbox* and the autonomous navigation pipeline powered by Nav2 and AMCL.
- **Chapter 5: Map Partition and Collect Algorithm** Presents the core theoretical innovations: the Watershed-based Generalized Voronoi Partitioning algorithm and the hierarchical Behavior Tree for fault-tolerant block collection.
- **Chapter 6: Field Experiment and Analysis** Provides empirical validation of the system's mapping, localization, navigation, and map-partitioning capabilities in a real-world domestic environment.
- **Chapter 7: Conclusion** Summarizes the project's achievements and outlines future work.

Chapter 2

Hardware Architecture

This section illustrates the high-level overall architecture of the autonomous building block classification robot, which is divided into two core parts: the physical hardware architecture and the logical ROS software layered communication architecture.

2.1 Experimental Hardware Components

The hardware platform used in this experiment consists of the RoboMaster EP mobile chassis, an onboard computing unit, a 2D LiDAR sensor, connection materials, and 3D-printed structural components. These components provide the basic hardware support for autonomous navigation, building block collection, and system expansion.

2.1.1 RoboMaster EP Mobile Robot Chassis

The RoboMaster EP is used as the mobile base of the system, providing the chassis motion capability required for indoor navigation and block collection. The chassis uses four **Mecanum wheels**, enabling omnidirectional movement, including forward and backward motion, lateral translation, diagonal movement, and in-place rotation. This mobility allows the robot to adjust its position flexibly when approaching target regions or avoiding obstacles.

However, Mecanum-wheel odometry is sensitive to wheel slip, floor unevenness, and lateral motion errors. Therefore, odometry filtering and careful SLAM parameter configuration are required in real-world mapping and navigation experiments.

The RoboMaster EP also provides the original manipulator, camera module, motor control system, and expansion interfaces. These features make it suitable for secondary development, but the original mechanical structure is not sufficient for efficient multi-block collection or for mounting additional SLAM hardware. Therefore, further structural modifications are required, as discussed in Section 2.2. For further technical details, refer to [SZ DJI Technology Co., Ltd.(2020)].

2.1.2 NVIDIA Jetson Orin NX Edge AI Computing Unit

The NVIDIA Jetson Orin NX module is used as the onboard computing unit. It provides the computational resources required for real-time perception, SLAM processing, navigation, and high-level task execution. In this project, it runs the ROS 2-based software stack and communicates with the RoboMaster EP, LiDAR driver, perception module, and navigation modules.

The Jetson Orin NX supports SLAM mapping, AMCL localization, Nav2 navigation, and YOLO-based building block detection. Its interfaces, including USB, Ethernet, HDMI, and M.2 expansion, provide the required connectivity for sensor data transmission, debugging, storage, and system expansion. Since the original RoboMaster EP does not provide reserved mounting space for this board or its power supply, additional mounting structures are required. For further technical details, refer to [NVIDIA Corporation(2023)].

2.1.3 YDLIDAR X3 PRO LiDAR

The YDLIDAR X3 PRO is used as the main environmental perception sensor for SLAM mapping and navigation. It is a 360-degree 2D LiDAR that provides planar scan data of the surrounding environment, including walls, furniture, and obstacles.

Since the block collection task is performed mainly on a 2D indoor plane, the LiDAR scan data is sufficient for occupancy grid map construction, localization, and obstacle-aware navigation. The sensor data is integrated into the ROS 2 system and used by *slam toolbox*, AMCL, and Nav2.

For reliable mapping and localization, the LiDAR must be mounted near the geometric center of the chassis and positioned high enough to avoid occlusion. Therefore, a custom elevated mounting structure is required, as described in Section 2.2. For further technical details, refer to [Manuals Plus(2022)].

2.1.4 Interface Cables and Power Connection Materials

Interface and power connection materials are used to connect the LiDAR, Jetson Orin NX, RoboMaster EP chassis, and other electronic modules. These include serial cables, USB extension cables, and power adapter cables. Stable data transmission, power supply, and cable routing are necessary for reliable operation during mapping, navigation, and block collection.

2.1.5 3D Printing Material

Bambu Lab PLA titanium-gray filament is used to fabricate the customized mechanical components. PLA provides sufficient printing accuracy and moderate strength for rapid prototyping.

The printed parts include the L-shaped pusher plate, bucket-style storage container, battery case, and elevated mounting structure for the LiDAR and control board. These components adapt the original RoboMaster EP platform to the block-collection task and support the scoop-and-push collection mechanism described in Section 2.2.

2.2 Task Challenges, Limitations of the Original Mechanism, and Proposed Design Changes

2.2.1 Key Task Challenges

The block-picking task in this experiment presents three main challenges:

1. **Picking efficiency requirement:** The robot must avoid a “one building block at a time” strategy. Instead, it should be capable of collecting multiple building blocks in a single operation. Therefore, a container for temporary building block storage is necessary.
2. **Accommodating diverse building block shapes and sizes:** Building blocks vary in shape (e.g., cubes, cylinders) and size (from 2 cm to 5 cm). The mechanical design must adapt to different geometries to ensure all blocks can be reliably retrieved.
3. **Area coverage and autonomous navigation:** The robot must use SLAM to build a map and use localization and navigation modules to traverse all reachable areas in the experimental field and finally return to the starting point.

2.2.2 Limitations of the Original Mechanical Structure

The original RobotMaster platform has several hardware constraints that hinder direct fulfillment of the above requirements:

1. **Limited manipulator motion:** The original manipulator can only move in a two-dimensional plane (extension/retraction and vertical lifting), resulting in a very restricted workspace. With the stock gripper, it is impossible to place collected building blocks into any auxiliary storage device. Moreover, a storage container cannot be mounted on the sides or rear because the manipulator cannot reach those positions.
2. **Insufficient cable length:** The data cables connecting the camera, manipulator, and main control board are too short. During manipulator movement, they cause tethering and severely reduce the effective range of motion.
3. **Lack of a mounting platform for the LiDAR:** Reliable SLAM mapping and navigation require a LiDAR sensor to be fixed at the vehicle’s geometric center, with its scanning plane higher than the car body to avoid occlusion. However, the original

car body has no elevated platform to accommodate such placement.

4. **No reserved space for electronic boards and power supply:** Running SLAM, navigation, and low-level control algorithms demands an additional embedded board (NVIDIA Jetson), which together with its power source must be securely mounted. The original chassis provides no dedicated space for fixing these components.

2.2.3 Targeted Hardware Modifications

To overcome the limitations listed above, the following modifications were implemented:

1. **Relocating the manipulator to resolve cable interference:** The manipulator was detached from its original mounting position, moved forward and reattached to the front part of the car body. This relocation effectively eliminates the cable-length problem, as the manipulator can now move without being restricted by cable tension.
2. **Adding a bucket-style storage container:** After the manipulator was moved forward, the front layout was redesigned to accommodate a bucket-style storage container. A custom-sized, bucket-style storage container was designed and fabricated using 3D printing, then fixed at the front of the chassis. The container is designed with a slight bevel at the entrance and sits flush with the ground to minimize resistance when blocks are swept in.
3. **Replacing the gripper with an L-shaped pusher plate:** The original gripper was removed and replaced with a 3D-printed L-shaped plate. Instead of grasping, the manipulator uses this L-shaped plate to sweep blocks from the ground into the bucket. Combined with the ground-level bucket and the beveled entrance, this method enables multiple building blocks of different shapes and sizes to be swept into the bucket in one operation.
4. **3D-printed battery case for the power supply:** A dedicated battery case was designed and printed for the upper rear part of the chassis, providing a secure housing for the power source (e.g., Li-ion battery pack) and preventing it from shifting during motion.
5. **Two-tier elevated structure for LiDAR and control board:** Two complementary 3D-printed parts were designed:
 - The first tier is a four-post elevated base whose mounting interface exactly matches the holes on the chassis, allowing firm and quick attachment.
 - The second tier is a combined bracket: its lower section holds the embedded control board, while the upper section provides a mounting seat for the LiDAR.

By installing this combined bracket onto the four-post base, the LiDAR is precisely positioned at the chassis's center, and its scanning plane is raised above the highest point of the chassis. This ensures an unobstructed field of view for SLAM mapping

and navigation.

2.3 Mechanical Modeling, 3D Printing, and Hardware Installation

This section details the CAD modeling, 3D printing, and physical installation procedures for the hardware modifications proposed in Section 2.2. It also presents the geometric parameters, materials, mounting poses, and TF tree of the newly added components.

2.3.1 CAD Modeling and 3D Printing

All custom-designed structural components (the bucket, four-post elevated base, combined bracket, L-shaped pusher plate, and battery case) were modeled using SolidWorks and exported as Collada (*.dae*) files for integration into the ROS URDF/XACRO description. The dimensions of each model were customized based on the actual geometric space and interface locations of the RoboMaster EP chassis.

2.3.2 Hardware Installation Procedure

The installation procedure follows the physical assembly order of the modified platform:

1. **Manipulator forward relocation:** The original manipulator, mounted at the mid-rear of the chassis, was detached and reattached to the front area, shifting its rotation center by 0.14 m in the $+x$ direction relative to the chassis base. This resolved the cable-length limitation and allowed the front collection mechanism to be arranged together with the bucket.
2. **Bucket mounting:** The bucket is connected directly to `base_link` via a fixed joint with translation offset $(0.31, -0.08, 0)$ m and Euler angles $(\pi/2, 0, -\pi)$ rad. The bucket is designed so that its lower edge is approximately flush with the ground, and the bevel faces forward.
3. **Battery case mounting:** The battery case is attached to the rear part of the first-tier elevated base (`base1_link`) with offset $(-0.1, 0, 0.045)$ m and rotation $(1.57, 0, 0)$ rad, orienting the case opening upward for easy access to the power supply.
4. **Four-post elevated base (base1) mounting:** The base is fixed centrally on the upper surface of `chassis_base_link` with a vertical offset $z = 0.1$ m and no rotation. The four feet are secured to the chassis using M3 screws.
5. **Combined bracket (base2) mounting:** The bracket is attached to the top of `base1` with offset $(0.03, 0, 0.16)$ m (relative to `base1_link`) and no rotation. The lower part holds the Jetson Orin NX control board, while the upper part provides the LiDAR mounting plane.

6. **LiDAR mounting:** The LiDAR is connected to the top of the combined bracket via an invisible mount (`lidar_mount_link`), placing the `laser_frame` close to the vehicle's central axis at a height above the highest point of the chassis.
7. **L-shaped plate installation:** The original gripper was removed and replaced with a 3D-printed L-shaped plate. The plate is attached to `arm_2_link` via a fixed joint with translation offset (0.13, 0.018, -0.25) m and Euler angles (1.5708, 0, -1.5708) rad. This design enables the manipulator to sweep blocks into the front bucket rather than grasping them individually.

2.3.3 Summary of Component Geometric and Inertial Parameters

Table 2.1 lists the key parameters of the major modified or added components, including visual mesh files, mesh scales, simplified collision dimensions, estimated masses, and materials defined in the XACRO files.

Table 2.1. Summary of hardware component parameters

Component	Visual mesh file	Scale factor	Collision model	Mass (kg)	URDF visual material
Bucket	bucket.dae	1.0	box: $0.1 \times 0.1 \times 0.05$	0.5	dark_plastic
Manipulator base	arm_base.dae	1.0	mesh collision	0.3	metal
Manipulator link 1	arm_1.dae	1.0	mesh collision	0.056	metal
Manipulator link 2	arm_2.dae	1.0	mesh collision	0.08	metal
Four-post base	base1.dae	0.001	box: $0.05 \times 0.05 \times 0.05$	0.05	red_marker
Combined bracket	base2.dae	0.001	box: $0.1 \times 0.1 \times 0.02$	0.1	led_light
Battery case	battery_box.dae	0.001	box: $0.08 \times 0.04 \times 0.02$	0.2	dark_plastic
L-shaped plate	L_plate.dae	0.001	box: $0.005 \times 0.005 \times 0.005$	0.0001	dark_plastic
LiDAR	lidar.dae	0.001	cylinder: diameter 0.05, height 0.05	0.15	metal

Note: Collision models are simplified representations used for simulation. The visual mesh files provide the detailed appearance of each component. Mass values are estimated from the physical prototypes.

2.3.4 Coordinate Transforms and TF Tree

The TF tree is defined by the parent-child relationships listed in Table 2.2.

Table 2.2. Parent-child relationships and joint types

Parent link	Child link	Joint type
base_link	chassis_base_link	fixed
base_link	bucket_link	fixed
chassis_base_link	base1_link	fixed
base1_link	battery_box_link	fixed
base1_link	base2_link	fixed
base2_link	lidar_mount_link	fixed
lidar_mount_link	laser_frame	fixed
chassis_base_link	arm_base_link	fixed
arm_base_link	arm_1_link	revolute
arm_1_link	arm_2_link	continuous
arm_2_link	L_plate_link	fixed

The precise translation (x, y, z) and rotation (roll, pitch, yaw) parameters for the key fixed joints are listed in Table 2.3.

Table 2.3. Key joint transform parameters (parent $\rightarrow$ child)

Parent link	Child link	Translation (m)	Rotation (rad)
base_link	chassis_base_link	(0, 0, 0.03465)	(0, 0, 0)
base_link	bucket_link	(0.31, -0.08, 0)	(1.5708, 0, -3.1416)
chassis_base_link	base1_link	(0, 0, 0.1)	(0, 0, 0)
base1_link	battery_box_link	(-0.1, 0, 0.045)	(1.57, 0, 0)
base1_link	base2_link	(0.03, 0, 0.16)	(0, 0, 0)
base2_link	lidar_mount_link	(-0.01, 0, 0.03)	(0, 0, 0)
lidar_mount_link	laser_frame	(0, 0, 0.105)	(0, 0, 0)
chassis_base_link	arm_base_link	(0.14, 0.0010384, 0.0906477)	(0, 0, 0)
arm_2_link	L_plate_link	(0.13, 0.018, -0.25)	(1.5708, 0, -1.5708)

The manipulator’s operational joints (`arm_1_joint`, `arm_2_joint`, etc.) are revolute or continuous about the y -axis, with limits and mimic relations defined in the XACRO files. The LiDAR frame `laser_frame` has its z -axis pointing upward and x -axis forward, conforming to the REP 105 convention.

2.3.5 Visualization in RViz2

Figure 2.1 and 2.2 shows the final assembled robot model as displayed in RViz2. All modified components (front bucket, L-shaped plate, elevated LiDAR mount, battery case, and relocated manipulator) are clearly visible, validating the XACRO description and the hardware installation.



Figure 2.1. Front view showing bucket and L-shaped plate



Figure 2.2. Rear view showing battery case and LiDAR mount

2.3.6 Summary

Through the described CAD modeling, 3D printing, and precise hardware installation, all newly designed components have been successfully integrated into the original RoboMaster EP platform. The parameterised XACRO description helps maintain kinematic and geometric consistency between simulation and the real robot, providing a reliable foundation for subsequent SLAM mapping, path planning, and block-picking control algorithms.

Chapter 3

ROS Communication and YOLO Perception

3.1 Vision and Ranging Sensors

To achieve autonomous block collection, the robot must reliably perceive its environment and target objects. This system utilizes a combination of a high-resolution camera for object detection and an infrared sensor for distance measurement.

3.1.1 Camera Selection and Setup

The DJI RoboMaster EP accessory camera is utilized to capture the visual feed necessary for building block recognition. It is connected to the RoboMaster development board via a dedicated connection cable and is securely mounted on the second degree of freedom joint of the mechanical manipulator, providing an optimal downward-facing view of the operational area. The core specifications of the camera are as follows:

- **Viewing angle:** 120°
- **Maximum photo resolution:** 2560×1440
- **Video resolution:** FHD 1920×1080 30p, HD 1280×720 30p
- **Maximum video bitrate:** 16 Mbps
- **Image format:** JPEG
- **Video format:** MP4
- **Image sensor:** 1/4 inch CMOS
- **Effective pixels:** 5 million
- **Operating environment:** -10°C to 40°C

3.2 Onboard Computing Requirements

This system adopts the NVIDIA Jetson Orin NX module as the core onboard computing unit. This platform is specifically designed for high-performance AI inference at the edge, capable of providing up to 100 TOPS (INT8) of computing power. This is sufficient to

handle the parallel processing of video streams, complex Convolutional Neural Network (CNN) inferences, and real-time object recognition tasks.

3.2.1 Hardware Specifications

- **Processor (CPU):** Equipped with an 8-core Arm[®] Cortex[®]-A78AE v8.2 64-bit processor featuring 2MB L2 + 4MB L3 cache, with a maximum clock speed of 2GHz. It is responsible for system logic scheduling and video stream decoding.
- **Graphics and AI Acceleration (GPU & DLA):**
 - *GPU:* Adopting the NVIDIA Ampere architecture, it integrates 1024 NVIDIA[®] CUDA[®] cores and 32 Tensor cores, supporting efficient parallel image processing.
 - *Deep Learning Accelerator (DLA):* Equipped with 2x NVDLA v2.0 engines to relieve the tensor operation pressure on the GPU and optimize inference latency.
- **Memory:** Equipped with 16GB 128-bit LPDDR5 memory, featuring a bandwidth of up to 102.4 GB/s, supporting the storage of large-scale feature maps and concurrent multi-tasking.
- **Storage:** The system relies on an NVMe SSD connected via the M.2 Key M interface (256GB or above recommended) to ensure high-speed I/O read and write for the operating system and video cache.
- **Power Management:** Supports adjustable power consumption modes from 10W to 25W. When performing high-performance YOLO recognition tasks, it is set to MAXN mode to achieve maximum computing power output.

3.3 ROS Communication

3.3.1 System Objectives

- **Master ROS 2 Communication Patterns:** Understand and implement the three core communication methods: Topics (Publish/Subscribe), Services (Request/Response), and Actions (Goal/Feedback/Result).
- **Implement Behavior Control Logic:** Develop state machines to control robot behaviors (Search, Approach, Manipulate) based on sensor data.
- **Enhance System Robustness:** Implement multi-frame debouncing and timeout mechanisms to handle sensor noise and signal loss.
- **Coordinate Multi-Node Systems:** Verify the interaction between a Vision Node, a Mobile Base, and a Manipulator.

3.3.2 Architectural Principles

This system utilizes a modular architecture where specific behaviors are encapsulated in independent ROS 2 nodes. The communication flow is as follows:

- **Topics** (`/vision/detections`, `/cmd_vel`): Used for high-frequency, continuous data streams. The `vision_msgs/Detection2DArray` provides real-time object coordinates, while `geometry_msgs/Twist` controls the robot's velocity.
- **Services** (`~/start`): Used for discrete, synchronous triggers. The Behavior Tree (BT) triggers the Trigger service to start specific tasks (Find, Approach, Arm Move).
- **State Machines**: Each node maintains an internal state (e.g., `IDLE`, `ALIGN`, `APPROACH`) to manage the lifecycle of the task.

3.3.3 Implementation Steps

1. Target Search Node (`find_node.py`)

- **Function**: Rotates the robot to search for objects.
- **Logic**:
 - Subscribes to the `/vision/detections`.
 - Upon receiving a Trigger request, it publishes angular velocity (`Twist`) to rotate.
 - *Debounce Mechanism*: To prevent false positives, the node requires `required_confirm_frames` (default: 4) of consecutive valid detections (confidence > `min_confidence`) before stopping.
 - *Result*: Stops rotation and publishes a "SUCCESS" status once the target is consistently locked.

2. Approach and Alignment Node (`approach_node.py`)

- **Function**: Aligns the robot with the target and drives forward.
- **Logic & State Machine**:
 - `IDLE` → `ALIGN`: Calculates yaw error based on the target X-coordinate and applies a P-controller (`yaw_kp`).
 - `APPROACH`: Drives forward (`forward_speed`) while maintaining alignment.
 - `EXTEND`: When the target is too close (Y-coordinate > threshold) or temporarily lost, it enters a "blind drive" mode for a set duration (`extend_dist`) to ensure the robot gets close enough for manipulation.
 - *Robustness*: Uses Exponential Moving Average (EMA) filtering on target coordinates to smooth jitter.

3. Arm Control Node (`arm_preset_node.py`)

- **Function:** Executes a sequence of arm movements.
- **Logic:**
 - *Action Client:* Uses the `MoveArm` action to control the robotic arm.
 - *Concurrency:* Uses `MultiThreadedExecutor` and `ReentrantCallbackGroup` to ensure that blocking action calls (waiting for arm movement to finish) do not freeze the ROS spinner or delay service responses.
 - *Sequence Execution:* Executes a configurable list of presets (e.g., home → forward → down) defined in the parameter files.

3.4 YOLO Training and Deployment

3.4.1 Algorithmic Principles

YOLO (You Only Look Once) is a single-stage object detection algorithm that achieves end-to-end real-time detection by dividing the image into grids and directly predicting bounding boxes and class probabilities. In this project, we utilize the YOLOv8 model. Compared to previous iterations, YOLOv8 optimizes the network structure (e.g., replacing C3 modules with C2f modules) and the loss function, significantly improving detection accuracy and inference speed on edge devices.

Core Training Process:

- **Data Input:** Resize images to a uniform size, perform normalization, and apply data augmentation.
- **Feature Extraction:** Extract multi-scale features through the Backbone network.
- **Detection Head:** Predict bounding boxes and classes across three feature layers.
- **Loss Calculation:** Combine classification loss, bounding box regression loss (CIoU), and object confidence loss.
- **Post-processing:** Apply Non-Maximum Suppression (NMS) to filter out duplicate detection boxes.

3.4.2 Training Steps

Data Preparation:

- **Data Collection:** Collected 335 images containing various building blocks, sourced from custom self-taken photos tailored to the robot's camera perspective.
- **Data Annotation:** Utilized the Roboflow tool to mark bounding boxes, generate JSON format labels, and convert them into the standard YOLO TXT format.

- **Dataset Split:** The dataset was divided following a 7:2:1 ratio into a training set (235 images), a validation set (67 images), and a test set (33 images).

Model Training Parameter Configuration:

- **Pre-trained Model:** yolov8n.pt (Nano version chosen for optimized edge inference)
- **Training Epochs:** 100
- **Batch Size:** 64
- **Image Size:** 640×640

3.4.3 Training Result

Upon completion of the training program, the optimal model weights were saved as `best.pt` for subsequent deployment.

3.5 ROS Vision Node Integration

To bridge the deep learning model with the robot's control system, a dedicated ROS 2 node (`brickpick_vision`) was developed. This node subscribes to the `/camera/image_color` topic, converts the ROS image messages to OpenCV format via `CvBridge`, and feeds them into the YOLOv8 model for real-time inference. The detection results, including bounding box coordinates and class confidences, are structurally packaged and published as a `Detection2DArray` message to the `/vision/detections` topic, enabling downstream navigation and manipulator control nodes to make informed collection decisions.

Chapter 4

SLAM and Navigation

The ability to perceive the environment and move autonomously is fundamental to the Robotic Building Block Collection System. This chapter details the implementation of the Simultaneous Localization and Mapping (SLAM) and autonomous navigation modules.

This chapter first introduces the concepts of SLAM and navigation. It then explains how these modules are integrated into our system. Finally, it describes how the system is validated in simulation and deployed in a real-world environment.

4.1 Brief Introduction

SLAM (Simultaneous Localization and Mapping) is used to construct a map of an unknown environment while simultaneously estimating the robot's pose within that map. The core challenge SLAM addresses is often described as a "chicken-and-egg problem": to know where you are, you need a map, but to build a map, you need to know where you are. SLAM solves this by enabling a device, using data from its onboard sensors (like cameras, IMUs, and sometimes depth sensors like Time-of-Flight (ToF)), to construct a map of an unknown environment while simultaneously determining its own position and orientation (pose) within that newly created map. This self-contained process enables inside-out tracking, meaning the device tracks its position in 3D space without needing external sensors or markers (like Lighthouse base stations).[\[VR/AR Wiki\(\[n. d.\]\)\]](#)

In our system, the role of **SLAM** is to map the environment. As mentioned previously in the hardware section, we use the **YDLIDAR X3 Pro**, and the block collection task is performed in a 2D indoor environment. Therefore, we utilize the integrated 2D laser SLAM package: *slam toolbox* to generate an **occupancy grid map**. During the mapping stage, timely sensor updates are required to build an accurate indoor map. Consequently, we utilize the `async online` mode for mapping. Rather than relying on SLAM for localization, we intend to use the **AMCL** (Adaptive Monte Carlo Localization) module from the Nav2 stack; therefore, we use *slam toolbox* to export the map as *.pgm* (**P**ortable **g**ray **m**ap) and *.yaml* files, which can then be loaded by Nav2, as shown in Figure [4.1](#).

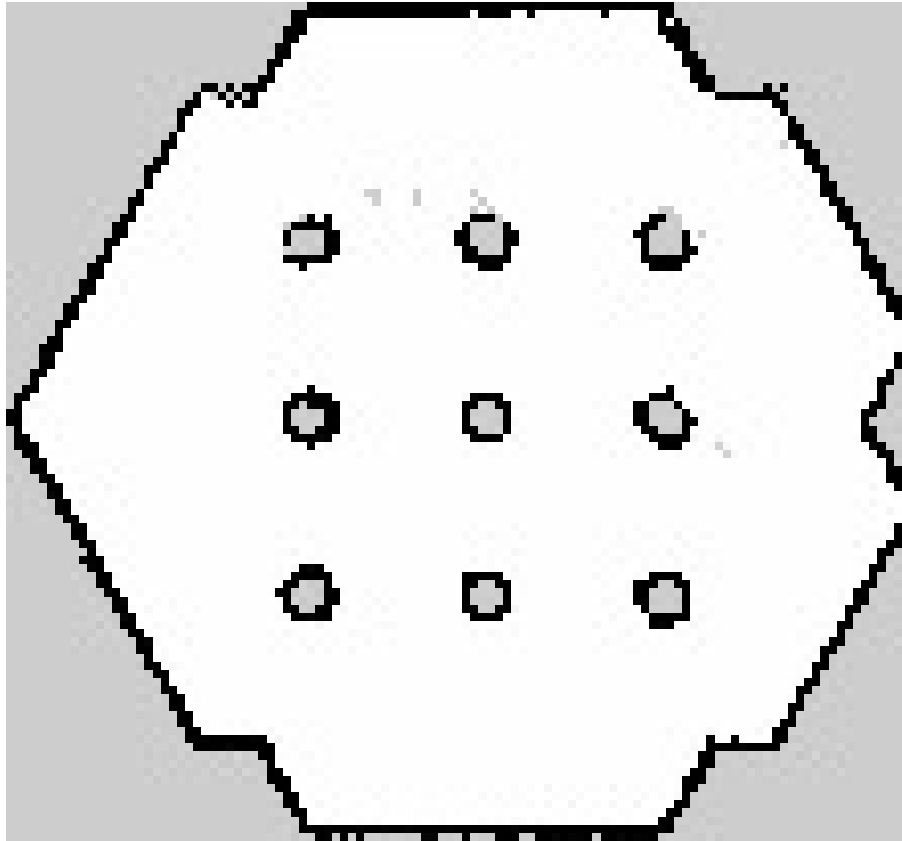


Figure 4.1. Saved occupancy grid map in PGM format

To implement our block-collection algorithm, we utilize **Nav2** for navigation. Nav2 is used as the navigation framework in this project because it provides localization, costmap generation, path planning, and obstacle avoidance functions for mobile robots. Nav2 uses **AMCL** to estimate the robot's pose on the pre-built map. Once an initial pose is provided, AMCL continuously tracks and optimizes the robot's position over time. Simultaneously, Nav2 generates **costmaps** to facilitate pathfinding and obstacle avoidance. We utilize the **MapsToPose** action service provided by Nav2 to command the robot to specific target coordinates.

4.2 Integration

4.2.1 Environment Integration

To run SLAM and navigation, several ROS 2 topics and transforms are required:

- **/scan**: Provides 2D laser scan data used to identify surrounding geometric features, such as walls and obstacles.
- **/odom**: Provides wheel-encoder-based odometry as an initial motion estimate for scan matching and localization.



Figure 4.2. USB connection

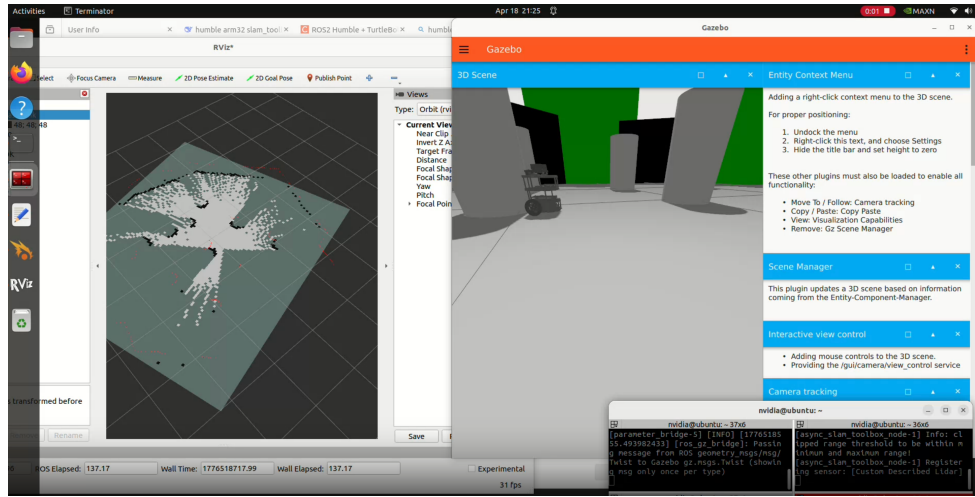
- **Static TF:** Defines the transform from `base_link` to `laser_frame`, allowing LiDAR data to be interpreted in the robot's base coordinate frame.

As shown in Figure 2.1 and 2.2, we developed a kinematic model for the robot and published the static transform between the LiDAR frame and the robot base frame. Simultaneously, the **RoboMaster EP** publishes its own wheel encoder data to serve as the **odometry source**.

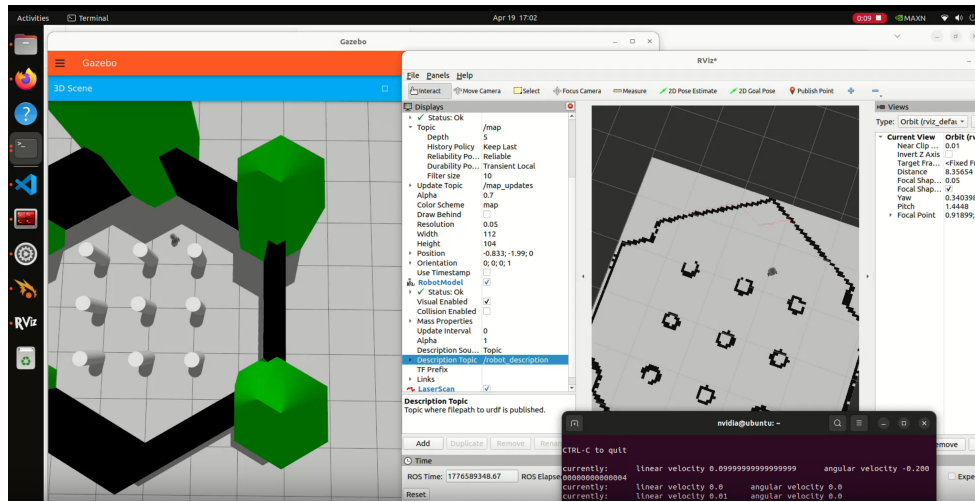
To obtain LiDAR data, the **YDLIDAR driver** is required to interface with the hardware. Subsequently, we must integrate it into **ROS 2** so that LiDAR data can be published to the corresponding topics. The necessary software is available as open-source on GitHub: [SDK](#), [Driver](#).

As shown in Figure 4.2, we connect the LiDAR to the development board using a data adapter and a USB data cable. The LiDAR used in this project is the **YDLIDAR X3 Pro**. Therefore, the serial port permission is granted, and the device port in the driver parameter file is configured correctly.

During the mapping stage, a controller is used to send velocity commands to the RoboMaster EP chassis, allowing the robot to move through the indoor environment while collecting LiDAR scan data for map construction.



(a) SLAM mapping process



(b) AMCL localization result

Figure 4.3. Simulation results of SLAM and navigation

4.3 Simulation

Since robot modeling, 3D printing, and hardware assembly are time-consuming, simulation was first used to verify the operation of SLAM and navigation. **TurtleBot3** was selected because it provides a complete set of ROS 2 packages and a stable simulation environment.

In the simulation environment, we verified the main workflow of map construction, map saving, AMCL localization, costmap generation, and target-point navigation.

Tip:

The Jetson Orin NX runs ROS 2 Humble. Since different TurtleBot3 branches support different Gazebo versions, the **gz-sim** branch was selected to match our simulation setup.

The TurtleBot3 world was selected for simulation because it is simple, widely used, and suitable for verifying basic navigation functions. As shown in Figure 4.3a, the SLAM module successfully generated the occupancy grid map. After the map was saved, the Nav2 lifecycle nodes were launched for localization and navigation.

As previously discussed, we utilize **AMCL** for localization; therefore, it must be initialized first. **Nav2** provides the necessary launch files, and we only need to pass the map information through a *.yaml* file. The robot's initial pose was then provided through the graphical interface in **RViz2**. As demonstrated in Figure 4.3b, the robot is able to estimate its pose within the map and continuously refine this pose during movement.

After localization is initialized, navigation can be executed. After the navigation lifecycle nodes are activated, a target pose can be provided, and Nav2 will plan and execute a path to guide the robot to the corresponding position. The next step is to deploy the verified SLAM and navigation workflow on the physical RoboMaster EP platform.

Chapter 5

Map Partition and Collect Algorithm

We have proposed a comprehensive **algorithmic pipeline** to drive a robot in collecting blocks across an entire area. Compared to traditional **Complete Coverage Path Planning (CCPP)**, this algorithm is significantly better suited to the specific task scenario and is expected to improve collection efficiency and reduce unnecessary movement compared with CCPP in terms of collection efficiency and power conservation. Therefore, the map is divided into several task-relevant open regions. The robot then inspects and clears these regions sequentially to reduce unnecessary coverage while avoiding missed blocks.

In this chapter, the algorithm is presented in two sections:

- **Map Partitioning:** We first introduce the Map Partition algorithm, the most critical preprocessing step in the pipeline. It divides the room map into several smaller sub-blocks that potentially contain blocks.
- **Collection Logic:** We then demonstrate how to implement the block collection workflow by leveraging the partitioned map in conjunction with Behavior Trees.

5.1 Map Partitioning

In this section, we will introduce our map segmentation algorithm in detail. Based on the assumptions that blocks are mainly located in open areas and tend to appear in clusters, full coverage of the entire map is unnecessary and inefficient. At the same time, since the system relies on YOLO-based visual detection, each inspection region should provide sufficient open space for camera observation and robot maneuvering. If there are obstacles in the area, it must be further divided into smaller and wider areas.

5.1.1 Pre-processing

According to **Assumption 3**, our manipulation targets (the building blocks) are exclusively located in open areas. Consequently, the vehicle is strictly constrained from entering overly narrow regions where collision risks are high or maneuvering is impossible. To establish a rigorous geometric and topological foundation for subsequent region parti-

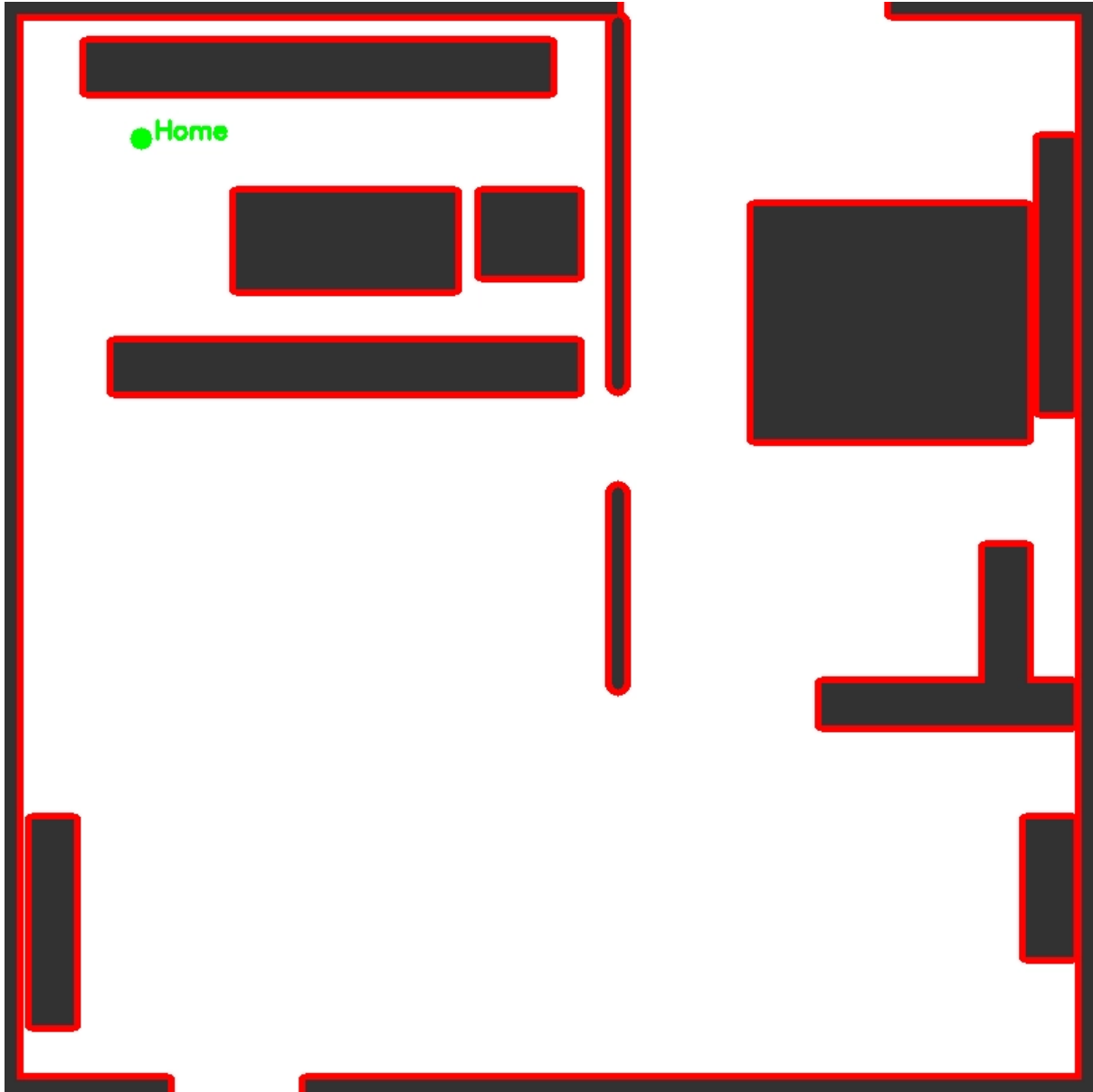


Figure 5.1. The inflation process

tioning, we transform the raw 2D map into a strictly constrained, safe navigation mask $\mathcal{M}$. This process involves three sequential operations:

Occupancy Discretization: Formally, starting from a ROS-compatible Occupancy Grid Map (OGM) [Moravec and Elfes(1985)], we apply a threshold τ to discretize the pixel probabilities into a binary representation: Free ($p = 0$), Occupied ($p = 1$).

Configuration Space (C-space) Inflation: As shown in Figure 5.1, to mathematically enforce the avoidance of "overly narrow areas," we adopt the Configuration Space concept [Lozano-Perez(1983)]. Let r_{rob} denote the minimum circumscribed radius of the vehicle's footprint. We perform a morphological dilation operation on the Occupied pixels using a disk-shaped structuring element with a radius of r_{rob} . This operation effectively computes the Minkowski sum between the obstacle map and the vehicle's

footprint. Consequently, any spatial corridor or gap with a physical width less than $2r_{rob}$ is entirely occluded in the dilated map. The remaining free pixels provide a conservative estimate of geometrically passable space for the vehicle.

Topological Reachability Filtering: While C-space inflation guarantees geometric safety, it does not account for topological accessibility. Grid maps often contain isolated free spaces (e.g., areas separated by walls, or SLAM artifacts) that are geometrically open but physically unreachable. To prevent the generation of target regions in these unreachable "islands," we apply a Connected Component Labeling (CCL) algorithm to identify all independent free-space clusters $\mathcal{C} = \{C_1, C_2, \dots, C_n\}$. We then extract the specific cluster C_{home} that contains the vehicle's initial starting point (Home).

The resulting binary mask $\mathcal{M} \subset \Omega$ (where $\mathcal{M}(x, y) = 1$ denotes safe and reachable space) serves as the strict spatial boundary for all subsequent steps. This ensures that any generated sub-regions and their corresponding navigation centers are topologically connected to the Home point and geometrically collision-free.

5.1.2 Seed Point Extraction

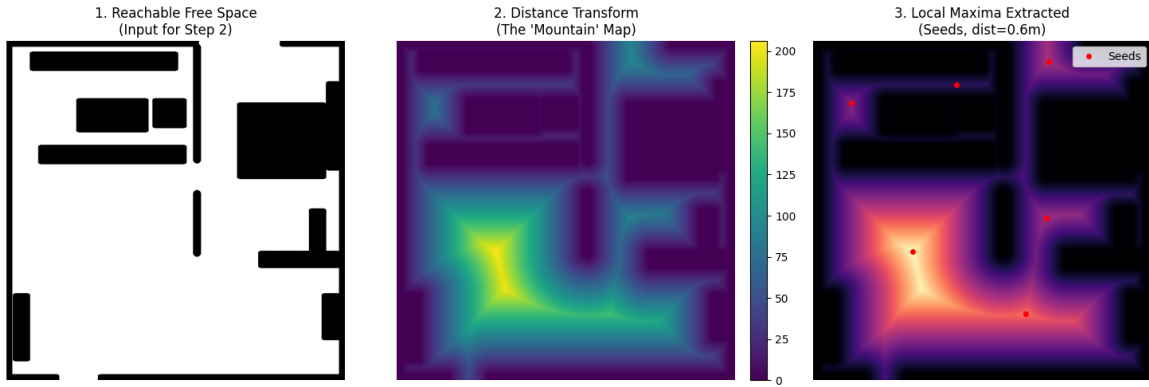


Figure 5.2. Distance transform and seed point extraction

According to **Assumption 2**, building blocks are generally clustered in open areas rather than distributed uniformly near obstacles. To computationally anchor these open areas, we select local maxima in the distance field, which correspond to points that are far from nearby obstacles. This point is defined as the **seed point**. The collection of these seed points serves as the nuclei for subsequent Voronoi-based space decomposition.

To extract these seed points robustly and scale-invariantly, we employ a three-stage pipeline: Euclidean Distance Transform (EDT), physical-scale decoupling, and constrained non-maximum suppression.

Openness Quantification via Distance Transform: To mathematically evaluate how "open" a specific pixel is, we apply the Euclidean Distance Transform (EDT) [Felzenszwalb and Huttenlocher(2012)] to the preprocessed safe mask $\mathcal{M}$. For any free

pixel $\mathbf{p} \in \mathcal{M}$, the EDT calculates its minimum Euclidean distance to the nearest obstacle pixel $\mathbf{q} \in \Omega \setminus \mathcal{M}$.

$$D(\mathbf{p}) = \min_{\mathbf{q} \in \Omega \setminus \mathcal{M}} \|\mathbf{p} - \mathbf{q}\|_2 \quad (5.1)$$

This generates a continuous scalar field D_{map} , as shown in Figure 5.2.

Physical-Scale Decoupling: A critical challenge in grid-based mapping is resolution dependency. If we were to extract local maxima purely in the pixel domain, a high-resolution map would yield an excessive density of seed points, fragmenting the space, while a low-resolution map might miss crucial areas. To achieve resolution invariance, we introduce a physical constraint parameter d_{min}^{phys} (e.g., 0.6 meters), which represents the minimum acceptable physical distance between two distinct seed points. Given the map resolution ρ (meters/pixel), the pixel-level minimum distance d_{min}^{px} is derived as:

$$d_{min}^{px} = \left\lceil \frac{d_{min}^{phys}}{\rho} \right\rceil \quad (5.2)$$

This constraint prevents excessive seed density and keeps the partition scale consistent across maps with different resolutions.

Constrained Local Maxima Extraction: With the scalar field D_{map} and the scale constraint d_{min}^{px} established, we extract the seed set $\mathcal{S}$ using the `peak_local_max` function. In the following formulation, this constrained peak extraction process is denoted by the operator $\Phi(\cdot)$. This function detects local peaks in the distance-transform map while enforcing a minimum pixel distance between selected peaks. In this project, d_{min}^{px} is used as the minimum separation constraint, ensuring that the extracted seed points remain spatially distributed and do not become overly dense in large open areas.

Formally, the extracted seed set can be represented as:

$$\mathcal{S} = \Phi(D_{map}, d_{min}^{px}, \mathcal{M}) \quad (5.3)$$

where $\Phi(\cdot)$ denotes the constrained local-maximum extraction operator implemented by `peak_local_max`. The mask $\mathcal{M}$ restricts peak extraction to the safe and reachable region, while d_{min}^{px} enforces a minimum separation between selected peaks. These seed points are distributed across different open sub-areas and serve as representative centers for subsequent partitioning.

5.1.3 Voronoi Partitioning

After the seed points $\mathcal{S}$ are extracted as representative centers of open areas, our next step is to delineate the precise spatial boundaries of these areas. To achieve this, we partition the entire reachable free space $\mathcal{M}$ into distinct regions, where each region

exclusively corresponds to one seed point.

The Limitation of Standard Euclidean Voronoi: Conceptually, partitioning space by assigning every point to its nearest seed is a classic Voronoi tessellation problem [Okabe et al.(1992)]. For a set of seeds $\mathcal{S} = \{s_1, s_2, \dots, s_k\}$, a standard Voronoi cell V_i is defined as:

$$V_i = \{\mathbf{p} \in \mathbb{R}^2 \mid \|\mathbf{p} - s_i\|_2 \leq \|\mathbf{p} - s_j\|_2, \forall j \neq i\} \quad (5.4)$$

However, in a grid map containing non-convex obstacles, applying Eq 5.4 directly using Euclidean distance is fundamentally flawed. Two seed points separated by a wall might be geometrically close, causing the standard Voronoi boundary to slice directly through the wall, assigning inaccessible pixels to the wrong region.

Watershed-based Generalized Voronoi Partitioning: To resolve this, we require a **Generalized Voronoi Diagram (GVD)** [Choset et al.(2007)] that respects obstacle boundaries. Instead of computing distances point-by-point, we leverage a profound topological equivalence in image processing: the Watershed algorithm applied to a distance map can be used to obtain an obstacle-aware partition similar to a Generalized Voronoi Diagram.

We formulate the free space as a topographic surface $\mathcal{T}$, where the elevation at any pixel is defined as the negative of its distance transform value:

$$\mathcal{T}(\mathbf{p}) = -D_{map}(\mathbf{p}), \quad \mathbf{p} \in \mathcal{M} \quad (5.5)$$

Under this transformation, obstacle pixels are excluded from the flooding domain, while pixels near obstacles form relatively high boundaries in the transformed surface. (since $D_{map} \rightarrow 0$ implies $\mathcal{T} \rightarrow 0$), while the deepest "valleys" are located exactly at the seed points (where D_{map} is maximized, making $\mathcal{T}$ most negative).

We then use the seed points $\mathcal{S}$ as mandatory markers (sources). The watershed algorithm simulates "flooding" this topographic surface $\mathcal{T}$ from these markers. The water rises uniformly from all seeds simultaneously. The boundary between two adjacent regions R_i and R_j is formed exactly where the "water" from seed s_i meets the "water" from seed s_j .

Formally, a pixel $\mathbf{p}$ is assigned to region R_i if the minimal topographic effort path from any seed to $\mathbf{p}$ originates from s_i :

$$L(\mathbf{p}) = \arg \min_{s_i \in \mathcal{S}} \left(\inf_{\pi \in \Pi(s_i, \mathbf{p})} \max_{\mathbf{q} \in \pi} \mathcal{T}(\mathbf{q}) \right) \quad (5.6)$$

where $\Pi(s_i, \mathbf{p})$ denotes the set of all continuous paths from s_i to $\mathbf{p}$ strictly within $\mathcal{M}$. Because the flooding is constrained within $\mathcal{M}$ and cannot pass through obstacle pixels, the resulting partition matrix L naturally wraps around obstacles.

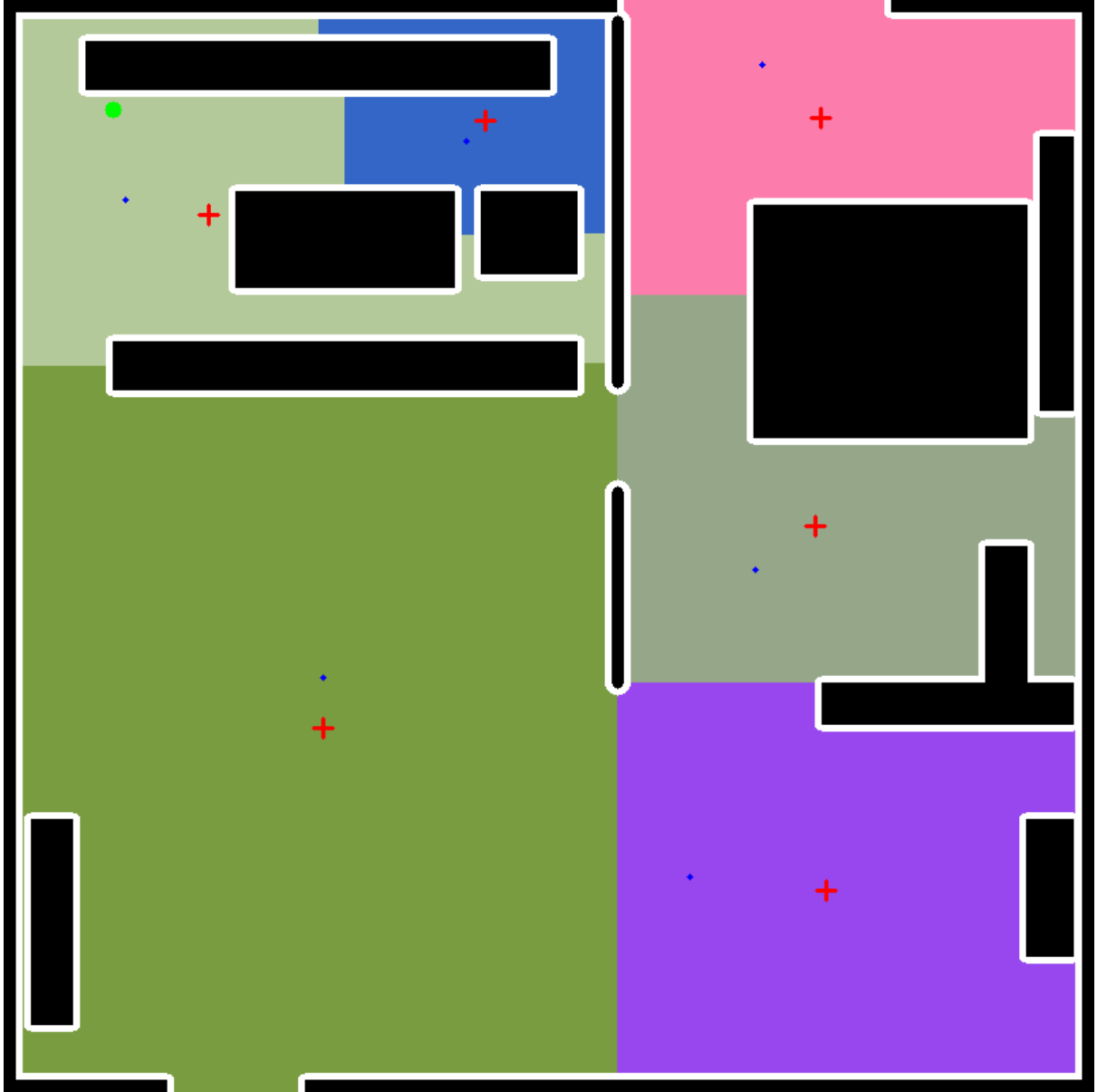


Figure 5.3. Watershed-based partitioning result

Resulting Morphology and Guarantees: This process yields a set of initial partitioned regions $\mathcal{R} = \{R_1, R_2, \dots, R_k\}$. By constraining the watershed transform to the pre-processed domain $\mathcal{M}$ and utilizing $-D_{map}$, as shown in Figure 5.3 we achieve four critical properties that are highly desirable for downstream visual-based collection tasks:

- **Obstacle Exclusion:** Because the transform domain $\mathcal{M}$ rigorously excludes all inflated obstacle pixels, any resulting region $R_i \subset \mathcal{M}$ contains no obstacle pixels in the processed navigation mask. This ensures that an agent deployed at a region's center operates in a purely open local field, inherently mitigating physical occlusions from environmental structures during visual sensing.
- **Obstacle-Induced Automatic Segmentation:** Unlike Euclidean Voronoi diagrams that erroneously bisect obstacles, the watershed "water" is physically blocked by the

high-elevation ridges formed by obstacles. Consequently, obstacles within a large space can act as natural boundaries and help divide the area into separate sub-regions, automatically slicing a single large area into distinct, isolated sub-regions.

- **Topological Connectivity:** Every region R_i is guaranteed to be topologically connected, forming a single contiguous cluster around s_i without fragmented islands.
- **Boundary Smoothness:** Because the flooding follows the gradient of the distance map, the boundaries tend to cut through the narrowest parts of the corridors between open areas (the saddle points of D_{map}), resulting in boundaries that better follow the structure of the free space.

5.1.4 Geometry Center

Finally, we can calculate the geometric center of the sub-region to serve as the candidate navigation waypoint. Let R_i denote the i -th finalized partitioned region. The initial candidate center $C_i = (y_i, x_i)$ is computed as the arithmetic mean of the pixel coordinates of all free space grids belonging to R_i .

However, due to the morphology-based obstacle inflation applied in the preprocessing stage to account for the robot's physical footprint (a 42cm chassis requiring an 8-pixel inflation radius), the mathematical centroid C_i might geometrically fall into an inflated obstacle boundary or an unreachable narrow bottleneck. Therefore, a strict reachability verification is mandatory. We utilize the A* pathfinding algorithm on the binary safe space mask $\mathcal{M}_{free}$ to query a collision-free path from the home coordinate P_{home} to C_i .

If a valid path exists, C_i is directly registered as the final reachable center. Conversely, if A* fails to find a path, a linear backoff correction mechanism is triggered to find a surrogate waypoint. We define the direction vector $\vec{d}$ pointing from P_{home} to the unreachable centroid C_i . We then sample discrete test points P_t along this vector line, starting from P_{home} and incrementally moving towards C_i . For each sampled point P_t , two conditions must be satisfied:

1. P_t must be clamped to the nearest valid grid in $\mathcal{M}_{free}$.
2. P_t must strictly remain within the spatial boundary of its original sub-region R_i (i.e., the grid label at P_t must still be i). This constraint is crucial to prevent assigning a waypoint that erroneously crosses a wall into an adjacent region's territory.

Once a point P_t satisfying both conditions is found, A* is executed again from P_{home} to P_t . The first P_t that yields a successful path is accepted as the surrogate center for R_i . If the linear search reaches the end without finding any valid reachable point, the sub-region R_i is flagged as an unreachable occluded island and is discarded from the final mission sequence to prevent navigation deadlock.

5.2 Collection Algorithm

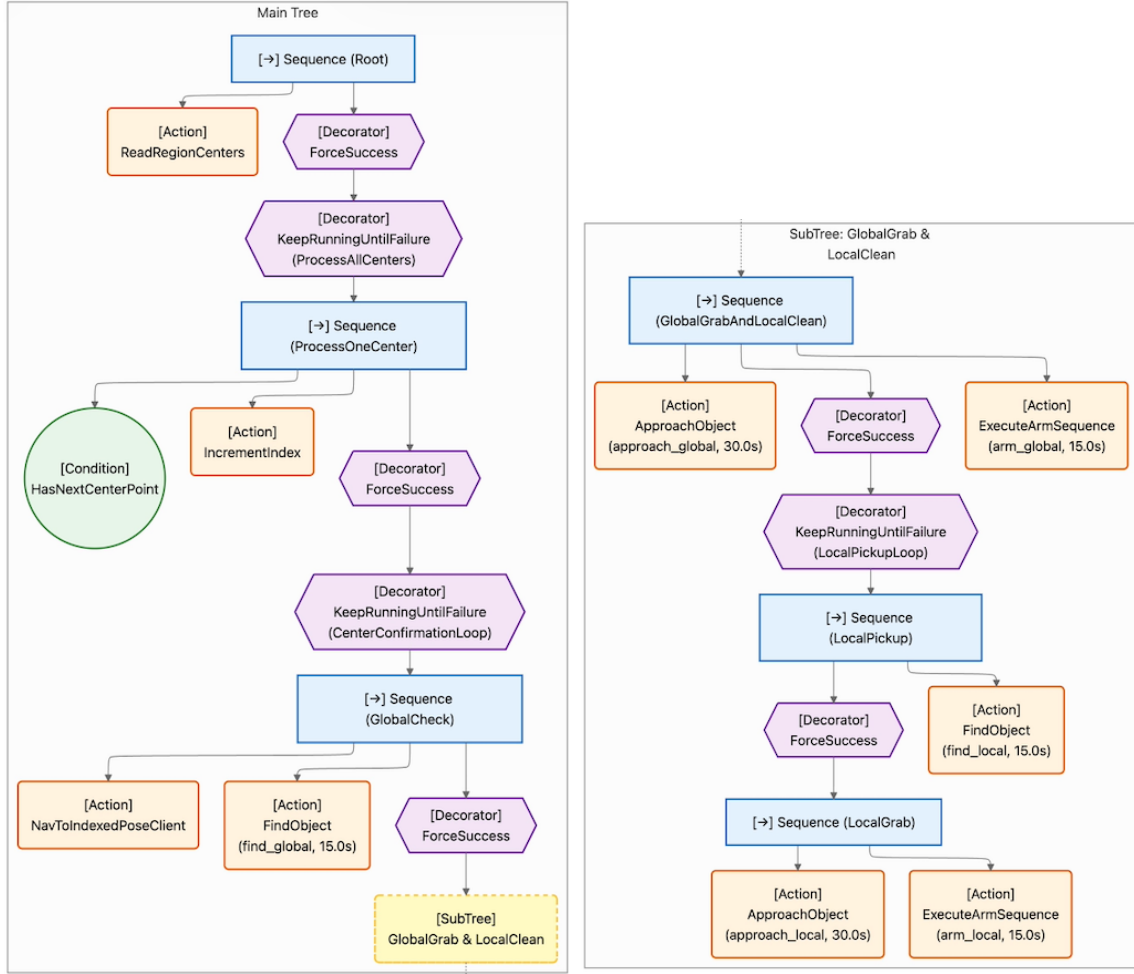


Figure 5.4. Behavior tree control flow structure for automated grasping and local cleaning tasks. This behavior tree (**MainTree**) defines the complete autonomous workflow of the system. The system first reads a preset list of regions and iterates through all center points via the **ProcessAllCenters** loop. At each waypoint, the robot navigates to the target pose, performs a global search (**GlobalCheck**), and executes a grasping action. Subsequently, it enters a local cleaning loop (**LocalPickupLoop**), continuously identifying and grasping surrounding objects until no targets remain in the field of view. Finally, it increments the index to proceed to the next region. *Note: In the diagram, blue rectangles represent control flow nodes (Sequence), purple hexagons represent decorator nodes (Decorator), orange rounded rectangles denote action executions (Action), and green ovals signify condition evaluations (Condition).*

In this section, we will focus on introducing our collection block algorithm. The algorithm introduced in Section 5.1 ensures that the region is divided into sub-regions containing all the building blocks. In simple terms, we just need to ensure that all the building blocks in each sub-area have been collected.

To achieve a highly autonomous, exhaustive, and fault-tolerant collection process, we designed a hierarchical Behavior Tree (BT) architecture, as depicted in Fig. 5.4. Compared to traditional Finite State Machines (FSMs), the BT offers superior modularity

and reactivity, which is crucial for handling dynamic disturbances in real-world physical environments. The core of our collection algorithm is built upon a **Hierarchical Center-to-Local Sweeping Strategy** integrated with a highly robust visual servoing pipeline.

5.2.1 Hierarchical Center-to-Local Sweeping Strategy

Blindly searching a vast area often leads to path redundancy and missed targets due to occlusions. To address this, our algorithm utilizes a nested loop structure within the behavior tree to guarantee absolute clearance of each sub-region:

- **Global Center Confirmation:** The robot first navigates to the absolute center of the current sub-region. It executes a 360-degree rotational scan to lock onto the most optimal target (typically the closest one in the lowest field of view) and executes the approach and pick-up sequence.
- **Local Continuous Cleanup:** Upon a successful grasp, the robot does not immediately return to the center. Instead, it triggers the `LocalPickupLoop`, performing an in-situ rotational scan to clear any adjacent blocks. This minimizes travel distance for clustered targets.
- **Strict Verification Mechanism:** Once the local scan yields no further targets (triggering a timeout failure in the `FindObject` node), the algorithm relies on the `ForceSuccess` decorator to elegantly absorb the failure. This mechanism forces the sequence to loop back, commanding the robot to navigate *back to the sub-region center* for another global check. The system only increments the index to proceed to the next sub-region when a full 360-degree scan from the absolute center confirms zero remaining blocks, thereby eliminating the possibility of missed targets due to temporary occlusions.

5.2.2 Robust Visual Servoing and Fault Tolerance

In physical deployment, visual noise and kinematic constraints often disrupt standard tracking algorithms. To ensure the reliability of the `ApproachObject` and `FindObject` action nodes, we implemented several industrial-grade robustness enhancements at the algorithmic level:

- **Multi-frame Debouncing & Confidence Filtering:** To reject false positives (e.g., shadows or reflections temporarily identified as blocks), the visual tracking node requires a minimum YOLOv8 confidence score (> 0.6) and mandates continuous target confirmation across multiple consecutive frames (e.g., 3 frames) before initiating the alignment state.
- **Exponential Moving Average (EMA) Filtering:** During the dynamic approach, the target bounding box coordinates are smoothed using an EMA filter. This dampens

the coordinate jitter inherent in real-time object detection, resulting in exceptionally smooth chassis steering and preventing oscillatory "bang-bang" control behavior.

- **Blind Zone Intelligent Takeover:** A critical challenge in visual servoing is the target escaping the camera's lower field of view as the robot gets intimately close. Our algorithm monitors the target's Y-coordinate threshold. If tracking is lost while the target is confirmed to be in the immediate lower proximity, the system intelligently transitions from visual tracking to a dead-reckoning "Blind Drive" (Extend) state. This ensures the block is pushed precisely into the collection mechanism without aborting the mission prematurely.

Through the combination of the exhaustive BT logic and the robust visual servoing nodes, the system demonstrates exceptional resilience. Even if a block is accidentally displaced during the approach, the `ForceSuccess` mechanism instantly catches the failure state, safely aborts the current arm sequence, and immediately restarts the search loop from the center, ensuring uninterrupted autonomous operation.

Chapter 6

Field Experiment and Analysis

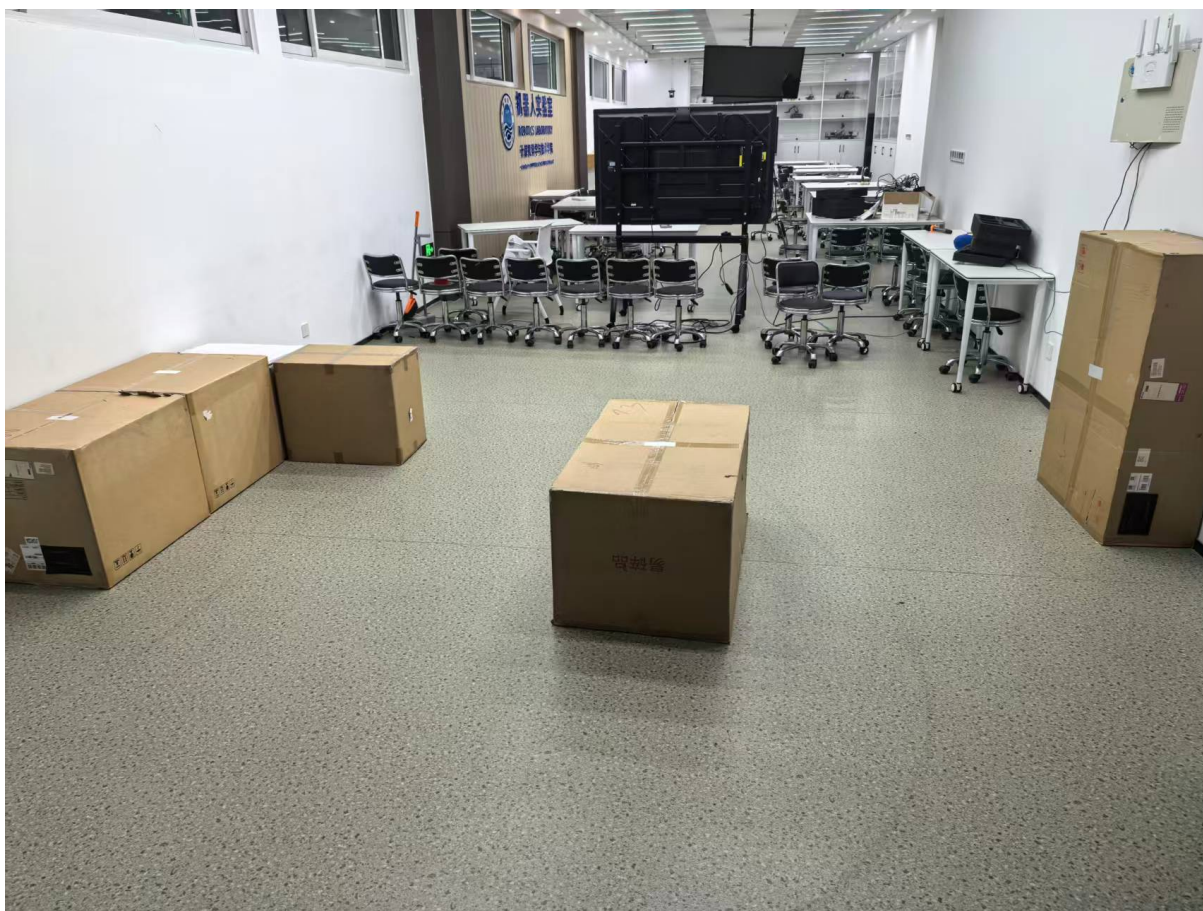


Figure 6.1. Experimental field layout. Boxes are used to represent furniture.

6.1 SLAM and Navigation

As described in Chapter 2, the robot uses **Mecanum wheels**, and the **YDLIDAR X3 Pro** provides 2D laser scan data. To obtain a reliable map in the real environment, several SLAM-related parameters and frame settings were adjusted. The robot was then tested in the living room environment shown in Figure 6.1.

6.1.1 Mapping

Due to the characteristics of **Mecanum wheels**, wheel odometry is sensitive to surface unevenness and wheel slip, which can introduce noticeable drift during mapping. To mitigate this issue, we employ a dedicated node to filter the raw wheel odometry data. Since the task is performed in a 2D plane, we use `base_link` as the main robot base frame and do not introduce a separate `base_footprint` frame.

The SLAM parameter file was duplicated and modified to match the frame names used in our robot model, including the odometry frame and the robot base frame. With the robot model and static LiDAR transform configured, the complete TF chain from `map` to `laser_frame` can be established. Now as we introduced in Chapter 3, we have a node to control the robot to move. We will use it to control the robot to move around in our living room. The remaining mapping procedure follows the simulation workflow described in the previous chapter.

6.1.2 Localization

After we build the map, we want to use AMCL to localize our robot. In Rviz we can give out the initial pose manually and control robot to move around. AMCL does not modify the map; instead, it estimates and continuously updates the robot's pose by matching LiDAR scan data with the pre-built map. Together, the mapping and localization results show that the SLAM and localization workflow works properly on the physical robot in the real-world environment.

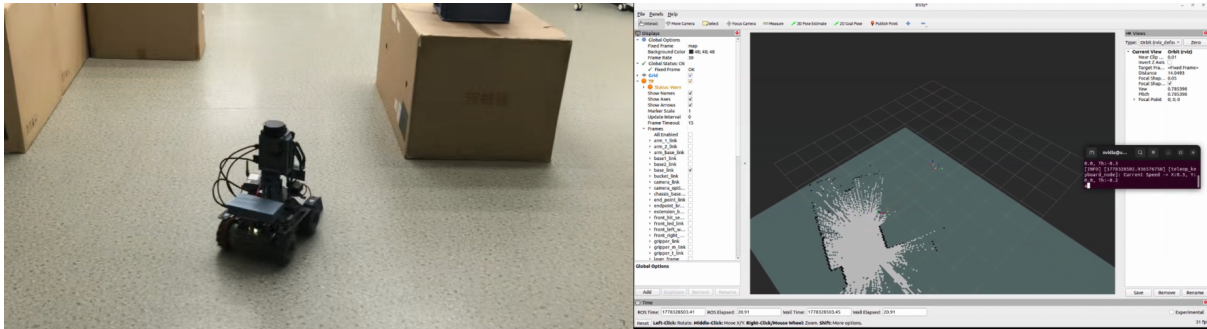


Figure 6.2. Mapping process in the real environment

6.1.3 Navigation

The real-world navigation workflow follows the same Nav2 pipeline as the simulation, but it must also handle sensor noise, odometry drift, and dynamic obstacles. We tested the `NavigateToPose` function by placing temporary dynamic obstacles in front of the robot. Nav2 was able to avoid these obstacles and update its path during navigation. It demonstrated the ability to dodge unmapped obstacles in real-time and re-plan a feasible

path once the obstacle was removed. Figure 6.3 illustrates the navigation visualization in RViz2; for a more detailed workflow, please refer to the demonstration video.

These experiments demonstrate that the robot can build a map, localize itself, and navigate in the real environment. This provides the necessary foundation for the subsequent autonomous block collection algorithm.

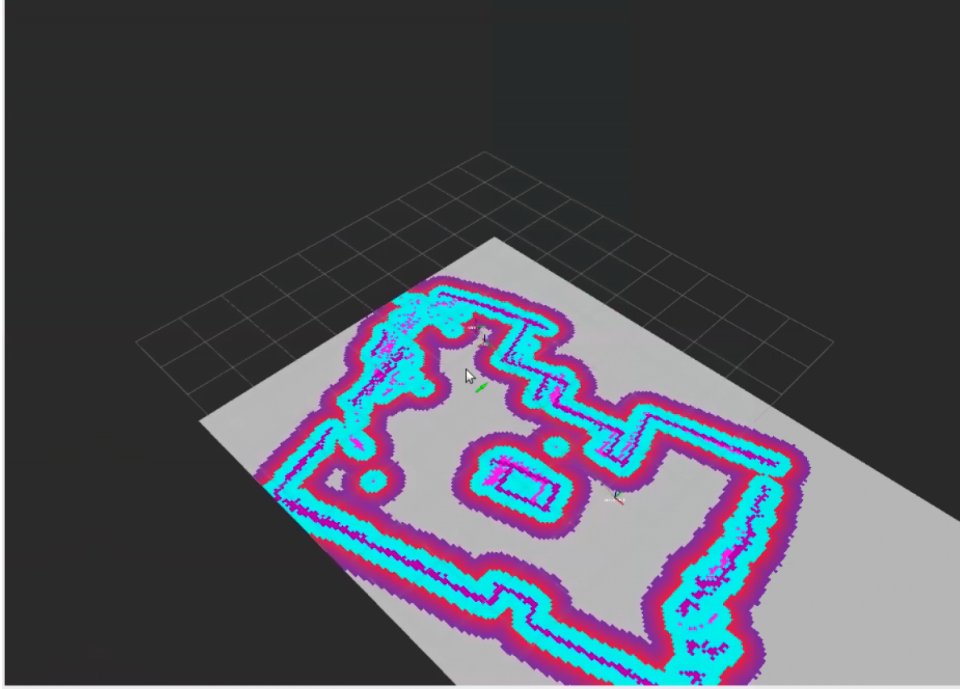


Figure 6.3. Navigation in our livingroom environment

The real-world experiment confirms that the SLAM and navigation modules can operate on the physical RoboMaster EP platform. However, the use of Mecanum wheels introduces odometry drift, especially on uneven surfaces. Therefore, odometry filtering and accurate LiDAR-frame calibration are important for maintaining mapping quality. In addition, the successful obstacle-avoidance test shows that Nav2 can handle temporary obstacles through local costmap updates and path re-planning.

6.2 Map partition

We apply our map partition algorithm on our real map. As shown in Figure 6.4, The six center coordinates are published in sequence along the optimal path and stored on the blackboard. The navigation nodes read the coordinates in sequence from it. Each sub-area performs the clearing block algorithm and then returns to the charging case.

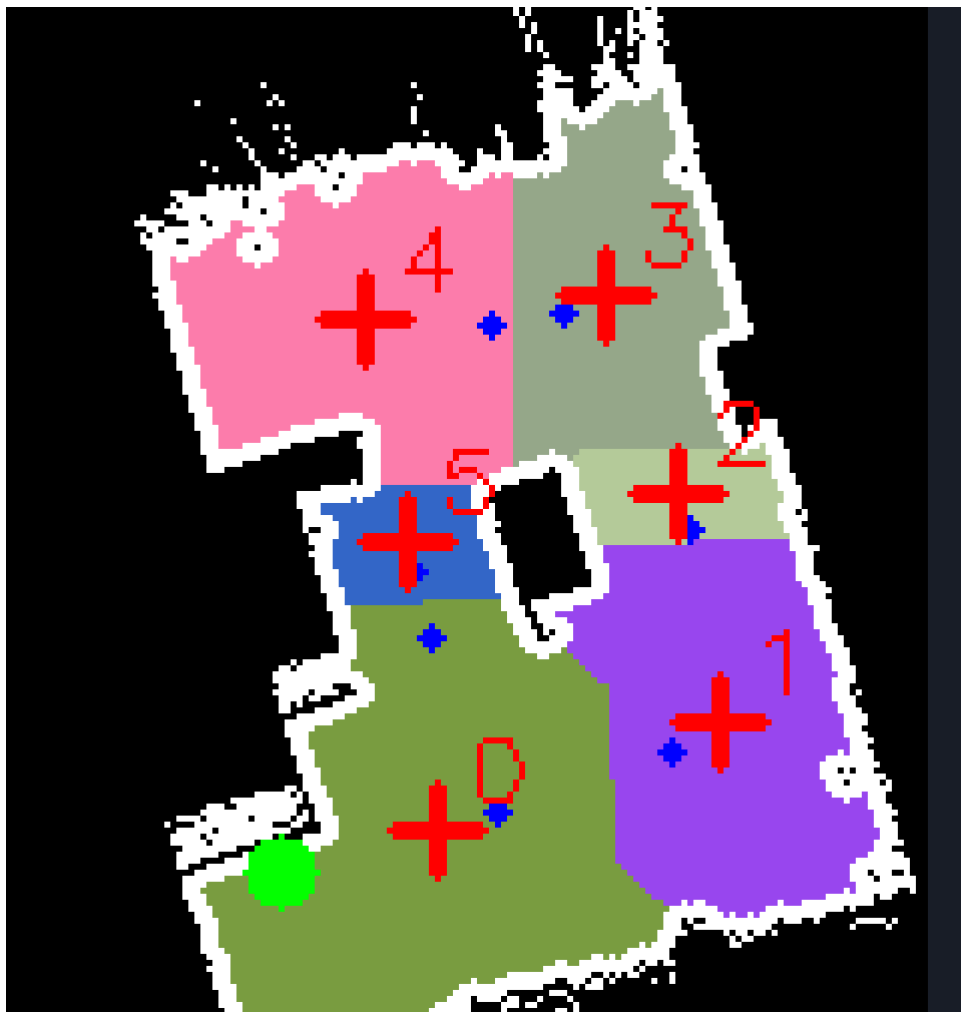


Figure 6.4. The partitioned map. The whole space is divided into 6 open spaces and each space guarantees the robot can capture the brick in it.

6.3 Analysis

Due to hardware constraints, the evaluation of the robotic system was divided into two primary experimental scenarios: "Single-Block across Multiple Regions" and "Multi-Block within a Single Region." Detailed visual demonstrations of these experiments are provided in the supplementary video materials.

6.3.1 Scenario 1: Single-Block across Multiple Regions

In the first scenario, individual building blocks were widely scattered across various corners of the testing environment. The primary objective of this test was to evaluate the system's global reliability in exhaustively identifying and collecting all scattered targets. Specifically, we assessed the visual recognition system's resistance to false positives, overall operational efficiency, the occurrence of missed targets, and the robustness of the behavior tree against unexpected execution deadlocks.

The experimental results demonstrated that the robot seamlessly executed the complete closed-loop cleanup cycle. It autonomously departed from the charging station, systematically navigated to and inspected each designated regional waypoint, successfully cleared the blocks, and flawlessly returned to the starting charging point. This performance validates the system’s exceptional global mapping and Nav2 path planning capabilities. It also proves that the robot can maintain high-precision object detection in complex environments and possesses a robust exception-handling mechanism to guarantee uninterrupted task execution.

6.3.2 Scenario 2: Multi-Block within a Single Region

The second scenario involved clustering multiple building blocks within a specific localized area while the robot executed the standard overarching cleanup workflow. This experiment was deliberately designed to stress-test the local collection efficiency when the system encounters high-density target zones.

During the trial, the robot fluidly transitioned into the local cleanup loop, continuously identifying and collecting adjacent blocks. Crucially, after clearing the immediate vicinity, it autonomously returned to the absolute center of the region to perform a final verification scan to check for any omissions. This behavior confirms that our hierarchical center-to-local sweeping algorithm is significantly more time-efficient than naive full-coverage (e.g., lawnmower) search patterns, drastically reducing path redundancy while strictly guaranteeing that zero blocks are missed.

Chapter 7

Conclusion

The development of the Robotic Building Block Collection System successfully addresses the practical challenges of autonomous indoor object retrieval. By modifying the DJI RoboMaster EP platform with custom 3D-printed hardware—including a bucket-style storage container and an L-shaped sweeping mechanism—the physical constraints of traditional single-grasp operations were overcome, significantly improving multi-block collection efficiency.

On the software front, the integration of the NVIDIA Jetson Orin NX and the ROS 2 framework provided a robust foundation for real-time perception and autonomous control. The system successfully utilized YOLOv8 for accurate block detection, alongside a 2D laser SLAM pipeline with Nav2 and AMCL for reliable environmental mapping and localization. These modules allowed the robot to effectively navigate indoor environments, handling dynamic obstacles and mitigating odometry drift.

A major contribution of this project is the innovative task-planning architecture. Instead of relying on conventional complete coverage path planning, a Watershed-based Generalized Voronoi Partitioning algorithm was implemented. This geometrically sound approach divided the safe, reachable space into optimal sub-regions based on physical openness. Combined with a hierarchical Behavior Tree control logic, the robot demonstrated a highly fault-tolerant “center-to-local” sweeping strategy. This ensured exhaustive block clearance with robust visual servoing enhancements, preventing mission failures due to temporary occlusions or visual noise.

Field experiments confirmed the system’s operational viability and resilience in real-world scenarios. Future work will focus on implementing automatic mapping for robots in unfamiliar environments, adding a user-friendly interface and further improving the robot hardware so that it can collect all the blocks in the room at once.

Bibliography

- [Choset et al.(2007)] Howie Choset, Kevin M. Lynch, Seth A. Hutchinson, George A. Kantor, Wolfram Burgard, Lydia E. Kavraki, and Sebastian Thrun. 2007. Principles of Robot Motion: Theory, Algorithms, and Implementation ERRATA!!!! 1. <https://api.semanticscholar.org/CorpusID:61848621>
- [Felzenszwalb and Huttenlocher(2012)] Pedro F. Felzenszwalb and Daniel P. Huttenlocher. 2012. Distance Transforms of Sampled Functions. *Theory Comput.* 8 (2012), 415–428. <https://api.semanticscholar.org/CorpusID:12212153>
- [Lozano-Perez(1983)] Lozano-Perez. 1983. Spatial Planning: A Configuration Space Approach. *IEEE Trans. Comput.* C-32, 2 (1983), 108–120. doi:10.1109/TC.1983.1676196
- [Manuals Plus(2022)] Manuals Plus 2022. *Product User Manual (ASIN: B0B4C49MLK)*. Manuals Plus.
- [Moravec and Elfes(1985)] H. Moravec and A. Elfes. 1985. High resolution maps from wide angle sonar. In *Proceedings. 1985 IEEE International Conference on Robotics and Automation*, Vol. 2. 116–121. doi:10.1109/ROBOT.1985.1087316
- [NVIDIA Corporation(2023)] NVIDIA Corporation. 2023. *Jetson Orin NX Series Data Sheet*. NVIDIA. <https://developer.nvidia.com/embedded/downloads> v0.5.
- [Okabe et al.(1992)] Atsuyuki Okabe, Barry Boots, and Kokichi Sugihara. 1992. Spatial Tessellations: Concepts and Applications of Voronoi Diagrams. In *Wiley Series in Probability and Mathematical Statistics*. <https://api.semanticscholar.org/CorpusID:117201421>
- [SZ DJI Technology Co., Ltd.(2020)] SZ DJI Technology Co., Ltd. 2020. *RoboMaster EP Programming Guide V1.0*. https://dl.djicdn.com/downloads/ROBOMASTER_EP/20200515/RoboMaster%20EP%20Programming%20Guide%20V1.0.pdf [PDF document].
- [VR/AR Wiki([n. d.])] VR/AR Wiki. [n. d.]. SLAM. <https://vrrarwiki.com/wiki/SLAM>. Accessed: 2023-10-24.